

SPOT CHECK

SKATESPOTS FIELD MANUAL · Nº01

CSS, LAYOUT & COMPONENTS

Everything we broke, debugged, and understood while building the SpotCheck nav bar — the hamburger, the SPOT CHECK title, the search pill, and the Add Spot button. Read it once; drill it with the demos and self-test.

12 chapters

Live demos

Self-test

Phase 2 — Static chrome

► CONTENTS (12 CHAPTERS + APPENDICES)

01 CSS Modules & how class names map

● BUG

● CONCEPT

KEYWORDS

CSS Modules

compound selector

hashed class name

— WHAT

We wrote `styles.title.white` and got `undefined`. The stylesheet had `.title_white` and a compound `.title.gold`, and none of it colored the words the way we expected.

— HOW

Prefer **single-purpose classes** over compound selectors:

```
/* .module.css */
.white { color: white; }
.gold { color: var(--color-accent); }
```

```
<span className={styles.white}>SPOT</span>
<span className={styles.gold}>CHECK</span>
```

► THE MENTAL MODEL — CSS IN, OBJECT OUT

YOU WRITE
(.MODULE.CSS)

```
.gold { color: gold
}
```



IMPORT GIVES YOU

```
styles = { gold:
  "SearchBox_gold__x7f2"
}
```



YOU USE

```
className=
{styles.gold}
```

— WHY

A CSS Module import is a **flat object of strings** — one key per class name, each value a uniquely-hashed class name. So: `styles.title` is a *string*; therefore `styles.title.white` reads a `.white` property off a string → `undefined`. A class named `title_white` becomes the key `styles.title_white`. A **compound selector** `.title.gold` matches only an element carrying *both* classes — CSS Modules makes two separate keys, and the rule fires only if both are present.

◆ IDEA TO CONSIDER

This bug class — "the object doesn't have the shape I assumed" — is the single most common source of `undefined` bugs in any codebase that generates objects from something else (CSS Modules, ORMs, API responses). Before debugging the logic, `console.log` the actual object and read its real keys.

02 next/font variable fonts: define once

● CONCEPT

KEYWORDS

CSS custom property

next/font

— WHAT

We needed Archivo Black across several components without repeating `archivo_black.variable` on every element.

— HOW

```
// layout.tsx – define the variable once at the root
<body className={archivo_black.variable}>{children}</body>
```

```
/* any .module.css – consume it anywhere below */
.white { font-family: var(--font-archivo-black); }
```

— WHY

The font is configured with `variable: "--font-archivo-black"`. That gives you a **CSS custom property**, not a ready-made font. `.className` applies a font directly to one element; `variable` only exposes the CSS variable — so you decide from CSS where it applies. Define it high, use it anywhere below.

◆ IDEA TO CONSIDER

This is a small instance of a general rule: something defined on an ancestor is available to every descendant automatically, via CSS inheritance/custom-property lookup. Fonts, colors, and spacing tokens all lean on that same mechanism — define once near the root, consume anywhere.

03 The OKLCH color model

● CONCEPT

— WHAT

We needed to read and tweak colors like `oklch(0.32 0.02 260)` and produce a "light grey."

— HOW

► SWEEP EACH CHANNEL

L — Lightness · 0 → 1 · dark → light

C — Chroma · 0 grey → vivid

H — Hue · 0–360° · red · yellow · green · blue · red

► THE APP'S ACTUAL TOKENS, RENDERED LIVE



— WHY

`oklch(L C H)` has three channels, each independent. `oklch(0.32 0.02 260)` = dark (L 0.32), nearly grey (C 0.02), faint blue (H 260) → a **dark blue-grey**. A light grey is high L, tiny C: `oklch(0.7 0.02 260)`.

◆ IDEA TO CONSIDER

OKLCH's whole selling point over hex/RGB is that its channels are *perceptually* independent — raising L always looks lighter, no matter the hue, which isn't true of HSL. That's why design-token systems increasingly standardize on it: predictable math for "make this 10% lighter" or "match this lightness at a different hue."

04 Design tokens

● BUG

● PATTERN

KEYWORDS

design token

:root

name by role

— WHAT

The same gold `oklch(0.78 0.19 85)` was hand-typed across four CSS files. Changing the brand color meant hunting every file.

— HOW

```
/* globals.css */
:root {
  --color-accent: oklch(0.78 0.19 85);      /* gold – brand accent */
  --color-bg-nav: oklch(0.21 0.014 260);
  --color-border: oklch(0.32 0.02 260);
  --color-text-muted: oklch(0.7 0.02 260);
  --color-text: white;
}
```

```
.gold { color: var(--color-accent); }
```

— WHY

A **token** is a named value defined once and referenced everywhere. `:root` is the top of the document, so variables declared there reach every component's CSS. Change the definition once → everything updates.

Name by role, not appearance. `--color-accent` beats `--color-gold` — if the color later changes to orange, the name still tells the truth.

◆ IDEA TO CONSIDER

This exact discipline scales up to entire design systems (and this manual's own palette is a token set). The verify trick: flip `--color-accent` to an obvious red and confirm

everything using it moves together. If one element doesn't, it's still hardcoded somewhere.

05 border-radius & the pill

● CONCEPT

KEYWORDS

border-radius

pill shape

— WHAT

Making a fully-rounded pill (search box, Add Spot button) that stays a clean pill at any width.

— HOW

```
.pill {  
  border-radius: 9999px;  
  padding: 10px 20px; /* height + room so the curve has space */  
}
```

► BOTH ARE LIVE — SAME MARKUP, DIFFERENT RADIUS

RADIUS 8PX

RADIUS 9999PX

— WHY

A **percentage** border-radius is relative to the element's own size, so a wide/short box becomes a stretched ellipse that shifts as it resizes. A large **fixed px** value clamps to a perfect half-circle on each end at any height. A pill needs height *and* horizontal padding — the curve needs room, and content must stay off the rounded ends.

◆ IDEA TO CONSIDER

Percentage vs. fixed-length is a recurring fork in CSS sizing generally (widths, radii, even font-size in some systems) — percentage always means "relative to something else," which is powerful but makes the result depend on context you may not control.

06 Which element owns what — the input saga

● BUG

● CONCEPT

KEYWORDS

`:focus-visible`

`caret-color`

`flex: 1`

Five separate symptoms, one theme: *a CSS property lives on a specific element, and native inputs ship with their own defaults.*

— WHAT

The input "turned blue when clicked" even with a `.input:focus { outline: none }` rule already in place. Plus: an invisible caret, a caret sitting inside the rounded corner, and an input that wouldn't fill the pill.

— HOW

SYMPTOM	FIX
Blue ring despite the CSS rule	<code><input className={styles.input} /></code> — the class was never attached
Caret invisible	<code>caret-color: var(--color-text)</code>
Caret inside the rounded corner	horizontal <code>padding</code> on the wrapper
Input won't fill the pill	<code>flex: 1</code> on the input

► THE FINISHED PILL — CLICK IT, TYPE, THEN TAB TO IT FOR THE KEYBOARD RING

🔍 Search Spots...

— WHY

The blue ring was the browser's *default* input rendering — the class carrying the override CSS had never actually reached the element. The rest were each a native-input default living on a specific, easy-to-forget property: caret color defaults to text color, native inputs have a fixed default width, and so on.

Debugging lesson: if styles "aren't applying," first confirm the class is actually on the element (inspect the DOM) before touching the CSS. Also: never remove `:focus` with nothing in its place — keyboard users rely on it. Prefer `:focus-visible` (keyboard focus only, not mouse clicks).

◆ IDEA TO CONSIDER

"Which element owns this property" is a question worth asking before writing any CSS rule at all — background, text color, caret, and focus behavior each have one specific, correct owner. Most "the CSS isn't working" bugs are actually "I'm styling the wrong element" bugs in disguise.

07 CSS needs units (and the last-property trap)

● BUG

KEYWORDS

invalid declaration

unitless length

— WHAT

`padding: 10 16px;` silently applied no top/bottom spacing.

— HOW

```
padding: 10px 16px; /* both values need units; end with ; */
```

— WHY

Any non-zero CSS length **requires a unit**. `10` (unitless) is invalid, so the browser discards the *entire* declaration. Related: a missing semicolon on the last property "works" until you add a line after it — which then silently gets swallowed too.

Invalid CSS fails silently. DevTools shows the struck-through declaration — train your eye to spot it.

◆ IDEA TO CONSIDER

CSS's error handling philosophy is "ignore what you don't understand, keep going" — great for browser compatibility across decades, brutal for debugging, since nothing ever throws. Get comfortable scanning DevTools' Styles panel for strikethrough, not just reading your source file.

08 padding vs gap

● CONCEPT

KEYWORDS

padding

gap

flexbox

— WHAT

The icon and input text in the search pill sat too close together.

— HOW

► SAME TWO CHIPS — LEFT USES PADDING, RIGHT USES GAP

icon input

padding: 16px /* space at the edges */

icon

input

gap: 16px /* space between children */

— WHY

padding = space *inside* an element's edges. **gap** = space *between* flex/grid children. Different jobs. The search pill used both: `gap` to separate the icon from the input, `padding` to keep everything off the rounded ends.

◆ IDEA TO CONSIDER

Before `gap` existed, spacing between flex children required margin tricks on individual children (and cleanup at the edges). One property replacing a whole category of margin hacks is a good example of CSS closing a real, long-standing gap in the language.

09 Icons: vs inline SVG vs library

● CONCEPT

KEYWORDS `currentColor` `tree-shaking`

— WHAT

Adding the search icon — three real options, with different trade-offs.

— HOW

```
import { Search } from "lucide-react";
<Search className={styles.icon} />
```

— WHY

OPTION	RECOLOR WITH CSS?	EXTRA REQUEST?	VERDICT
<code></code>	No	Yes	Bad for UI icons
Inline <code><svg></code>	Yes (currentColor)	No	Good, but verbose
Icon library (lucide-react)	Yes	No	Chosen — clean & tree-shakeable

The trade-off — recoloring + no extra request — recurs for every icon decision in this project.

◆ IDEA TO CONSIDER

This same three-way trade-off (asset file vs. inline markup vs. a maintained library) shows up constantly beyond icons — fonts, animations, even whole UI components. The library option usually wins once a project is bigger than a prototype, precisely because someone else already solved the recoloring/sizing/accessibility edge cases.

10 var() does NOT work in SVG attributes

● BUG

● GOTCHA

KEYWORDS

SVG presentation attribute

currentColor

— WHAT

```
<Search color="var(--color-text-muted)" />
```

 didn't apply the color.

— HOW

```
<Search className={styles.icon} />
```

```
.icon { color: var(--color-text-muted); } /* stroke = currentColor inherits this *
```

— WHY

lucide's `color` prop becomes an SVG **attribute** (`stroke="var(...)"`). CSS custom properties resolve only inside **CSS property values**, never inside raw XML/SVG presentation attributes — so the attribute stays an unresolved literal. Letting the icon default to `currentColor` and driving it from a CSS `color` rule works, because `currentColor` is a CSS value.

Remember it as one line: "var() lives in CSS, not in attributes." This bites with any SVG library, and again later with data-URI SVGs (Sheet 03).

◆ IDEA TO CONSIDER

The general rule underneath this: CSS custom properties only resolve where the browser evaluates *live CSS on the live DOM*. Anything static or generated separately from that — an SVG attribute, a data URI, a value baked at build time — can't see them. Worth checking for this pattern whenever a `var()` "just doesn't work."

11 React props: pass, receive, render, type

● BUG

● CONCEPT

KEYWORDS

JSX expression

prop typing

— WHAT

`<button>name</button>` rendered the literal text "name" instead of the value of a variable.

— HOW

PASS (PARENT)

```
<Button name="Add Spot" />
```



RECEIVE (CHILD)

```
function Button({ name  
}: ButtonProps)
```



RENDER

```
<button>{name}  
</button>
```

```
type ButtonProps = { name: string };  
function Button({ name }: ButtonProps) { ... }
```

— WHY

Bare text in JSX is a string. To evaluate JavaScript, wrap it in **braces**: `{name}`.

Untyped props are implicitly `any`; describing the shape means a required prop makes

TypeScript warn if a caller forgets it.

◆ IDEA TO CONSIDER

This full loop — pass, type, receive, render — is the entire mental model for every React component you'll ever build, no matter how complex. Every later chapter in this manual (`variant` , `level` , `grow`) is just this same loop with a more interesting value flowing through it.

12 Layout: shrink-to-fit & push-right

● CONCEPT

KEYWORDS `block-level` `justify-content` `margin-left: auto`

— WHAT

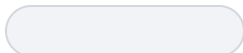
A `<div>` stretched full width when it should hug its content, and the Add Spot button needed to sit at the far right of the nav.

— HOW

► THE REAL NAV LAYOUT — LEFTGROUP + SPACE-BETWEEN



SPOT **CHECK**



ADD SPOT

► SPOTCHECKNAV HIERARCHY

```
<nav .div> justify-content: space-between | <div .leftGroup> display:flex;  
gap:16px | | <HamburgerButton /> | | <div .title> SPOT · CHECK </div> | |  
<SearchBox /> | <Button name="Add Spot" />
```

— WHY

A `<div>` is **block-level** — it fills 100% of its parent regardless of content; `width: fit-content` or `display: inline-block` makes it hug content instead. Horizontal position is decided by the **parent's** layout, not the element. Three flex tools solve push-right: `margin-left: auto`, `justify-content: flex-end`, or grouping + `space-between` — chosen here, since it keeps the reusable `<Button>` free of nav-specific positioning.

◆ IDEA TO CONSIDER

Grouping (wrapping the "other" items so the parent only ever has two children) is a small instance of a big idea: push layout responsibility up to the smallest common container, not onto individual leaf components. The same instinct shows up later as "layout props belong to the consumer, not the component" (Sheet 04).

A Glossary

CSS Module

A `.module.css` file whose class names get uniquely hashed and exposed as a plain JS object on import — one key per class name.

Compound selector

A CSS selector like `.title.gold` that matches an element only if it carries *every* listed class at once.

CSS custom property

A variable declared with `--name: value`, consumed elsewhere via `var(--name)`. Inherits down the DOM from wherever it's defined.

oklch()

A CSS color function with three perceptually-independent channels: Lightness, Chroma (saturation), Hue.

Design token

A named, single-source value (usually a CSS custom property) referenced everywhere instead of being hand-typed repeatedly.

:focus-visible

A CSS pseudo-class matching only keyboard-driven focus, not mouse clicks — lets you style a focus ring without showing it on every click.

currentColor

A CSS keyword resolving to the element's own `color` value; how SVG icons commonly inherit color from surrounding CSS.

Block-level element

An element (like `<div>`) that fills its parent's width by default, regardless of its content's actual size.

B Self-test

Answer aloud *before* opening each — explaining the "why" is what makes it stick.

Q1 Why is `styles.title.white` always `undefined` ? ▶

Q2 What does the `variable` font option give you, and where must its class be applied? ▶

Q3 In `oklch(0.7 0.02 260)`, which number makes it lighter? Which makes it greyer? ▶

Q4 Why did the input show a blue ring even though `.input:focus { outline: none }` existed? ▶

Q5 Why did `padding: 10 16px` do nothing? ▶

Q6 Difference between `padding` and `gap` ? ▶

Q7 Why didn't `color="var(--color-text-muted)"` work on the lucide icon? ▶

Q8 Three ways to push one flex item to the far right — and why is grouping cleanest here? ▶

Q9 Why name a token `--color-accent` instead of `--color-gold` ? ▶

C Study links — go deeper

— CORE DOCS

- ↗ [MDN — CSS reference](#)
- ↗ [React — Learn](#)
- ↗ [Next.js — docs](#)
- ↗ [TypeScript — Everyday Types](#)

— LAYOUT & COLOR

- ↗ [CSS-Tricks — Complete Guide to Flexbox](#)
- ↗ [Flexbox Froggy — play to learn flexbox](#)
- ↗ [oklch.com — color picker](#)

SPOT CHECK

SkateSpots Field Manual N°01 · built from the SpotCheck nav-bar session · keep it, drill it, then delete the crutch.